HUTECH University

TIỂU LUẬN MÔN CƠ SỞ DỮ LIỆU

XÂY DỰNG HỆ THỐNG BÁN HÀNG TRANG SỨC VỚI POSTGRESQL

Nhóm 6

Phùng Anh Tuấn – 2541864024
Võ Bùi Duy Thông – 2541864019

GVHD: TS. Lê Thị Ngọc Thảo · Lớp 25SCT41 · Tháng 10/2026

Nội dung

1. Vì sao chọn PostgreSQL
2. Kiến trúc hệ thống
3. Thiết kế CSDL: lược đồ 8 bảng và các loại index
4. 6 tính năng nổi bật gắn với nghiệp vụ
5. Ứng dụng, demo và benchmark trên 1 triệu đơn hàng
6. Kết luận

Vì sao chọn PostgreSQL?

#4
DB-Engines, sau Oracle, MySQL, SQL Server

+21,97
điểm nửa đầu 2026 tăng mạnh nhất top 5

4 lần
DBMS of the Year 2017 · 2018 · 2020 · 2023

100%
mã nguồn mở, không có bản Enterprise thu phí

CSDL QUAN HỆ ĐẦY ĐỦ

- Giao dịch ACID, MVCC: đọc không chặn ghi
- Khoá ngoại, CHECK, UNIQUE, generated column
- DDL nằm trong transaction → migration an toàn
- Hàm PL/pgSQL, trigger: logic nằm sát dữ liệu

+ ĐA MÔ HÌNH TRONG CÙNG MỘT HỆ THỐNG

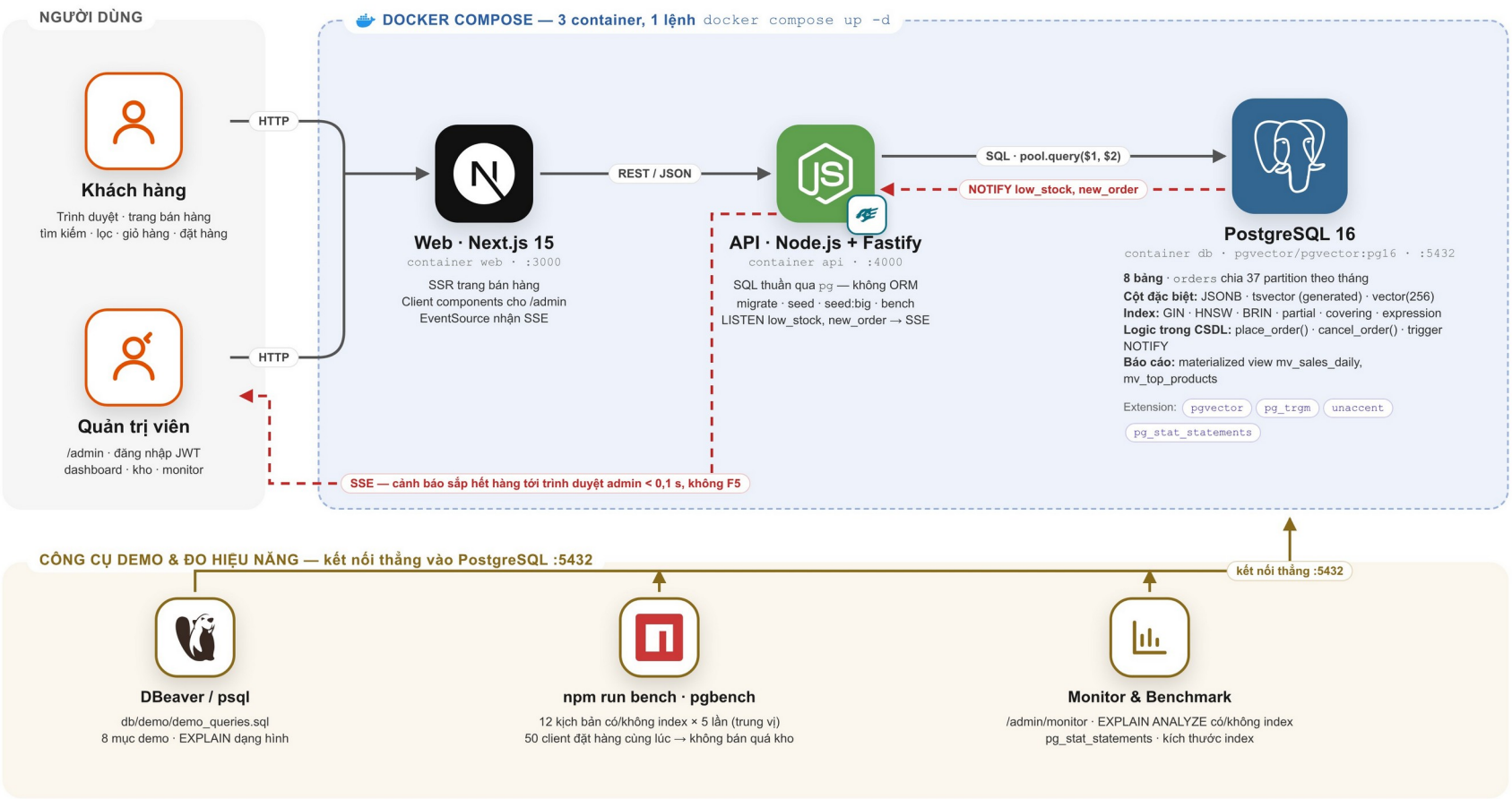
- JSONB, mảng, full-text search, pgvector
- 6 loại index: B-tree, Hash, GIN, GiST, SP-GiST, BRIN
- Partition, materialized view, LISTEN/NOTIFY
- Extension: pgvector, pg_trgm, unaccent, pg_stat_statements

Cách làm của nhóm 6: mỗi tính năng PostgreSQL gắn với 1 nghiệp vụ thật của cửa hàng trang sức và có số đo trên 1 triệu đơn hàng

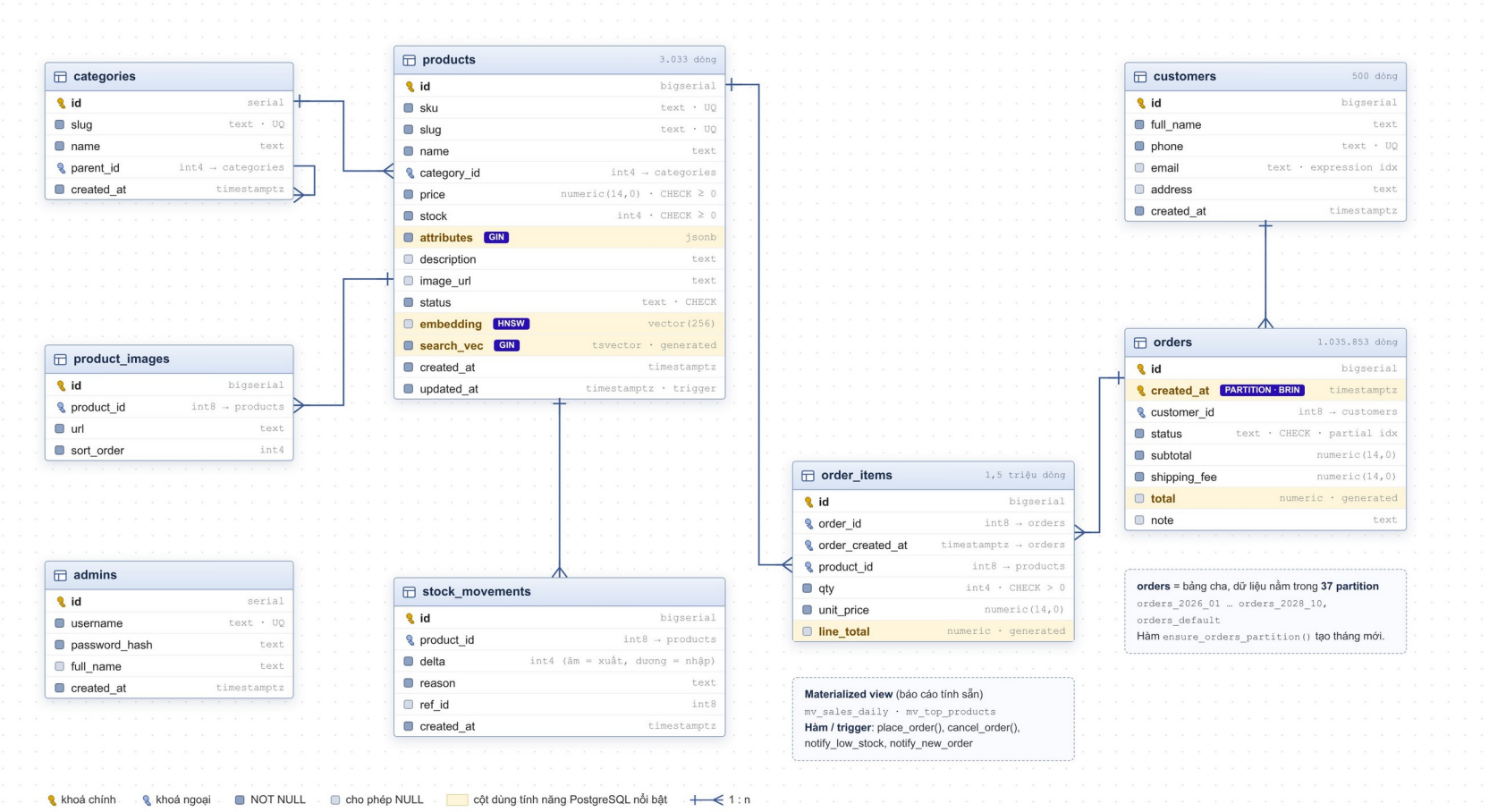
6 đặc thù nghiệp vụ – 6 tính năng PostgreSQL

1	Mỗi loại trang sức có thuộc tính khác nhau (ni, carat, dài dây…)	→	**JSONB + GIN index**
2	Khách gõ không dấu, sai chính tả vẫn phải tìm ra	→	**Full-text search + unaccent + pg_trgm**
3	Gợi ý “sản phẩm giống” khi xem chi tiết	→	**pgvector + HNSW**
4	Đơn hàng tăng mãi theo thời gian	→	**Partitioning theo tháng + BRIN**
5	Tồn kho ít, 2 khách mua cùng lúc chiếc cuối	→	**Transaction + SELECT … FOR UPDATE**
6	Dashboard nhanh, báo hết hàng tức thì	→	**Materialized view + LISTEN/NOTIFY**

Kiến trúc hệ thống



Thiết kế CSDL: 8 bảng



Thiết kế CSDL: các loại index đặc biệt

Loại	Trong hệ thống	Đo trên 1 triệu đơn
Partial index	`orders (created_at)` `WHERE status IN (pending, confirmed)`	0,2 ms so với 98 ms (480×) index chỉ 17 MB
Covering index (INCLUDE)	`orders (customer_id, created_at)` `INCLUDE (id, status, total)`	Index Only Scan, Heap Fetches = 0 0,2 ms so với 52 ms (250×)
Expression index	`customers (lower(email))`	Tìm email không phân biệt hoa thường
Generated column	`total = subtotal + shipping_fee` `search_vec = to_tsvector(...)`	CSDL tự tính, app không ghi sai được

Mỗi index gắn với đúng 1 truy vấn của ứng dụng. Theo dõi lượt dùng bằng pg_stat_user_indexes: index có idx_scan = 0 lâu ngày là ứng viên để xoá.

1. JSONB – thuộc tính sản phẩm linh hoạt

BÀI TOÁN

Nhẫn có ni, carat, nước màu; dây chuyền có chiều dài; vòng có đường kính…
Thêm cột → hàng chục cột NULL; bảng EAV → JOIN nhiều lần.

GIẢI PHÁP

```
attributes JSONB NOT NULL DEFAULT '{}'

CREATE INDEX idx_products_attributes
  ON products USING GIN
  (attributes jsonb_path_ops);

-- 1 câu SQL cho mọi tổ hợp bộ lọc
SELECT * FROM products
WHERE attributes @> '{"metal":"Vàng 18K",
                      "stone":"Kim cương"}';
```

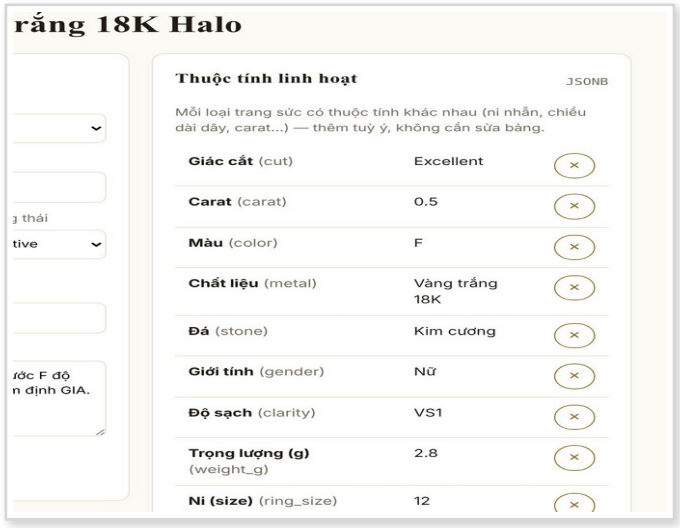
002 · 004_indexes.sql

- Thêm loại trang sức mới: không ALTER TABLE
- Bộ lọc chất liệu / đá trên web sinh từ JSONB

KẾT QUẢ

34×

nhanh hơn quét tuần tự
0,06 ms so với 2,1 ms



2. Full-text search tiếng Việt

BÀI TOÁN Khách gõ “nhan kim cuong 18k” (không dấu) hoặc “nhan kim cuog solitare” (sai chính tả) vẫn phải tìm ra đúng sản phẩm. LIKE không có index, không xếp hạng.

GIẢI PHÁP: UNACCENT + TSVECTOR + PG_TRGM

```
search_vec TSVECTOR GENERATED ALWAYS AS (
  to_tsvector('simple', vi_unaccent(name))
) STORED;

CREATE INDEX ... USING GIN (search_vec);
CREATE INDEX ... USING GIN
  (vi_unaccent(name) gin_trgm_ops);

-- Tầng 1: full-text, xếp hạng ts_rank
WHERE search_vec @@
      plainto_tsquery(vi_unaccent($1))
-- Tầng 2: sai chính tả → trigram
WHERE vi_unaccent(name) % 'nhan kim cuog'
```

001 · 002 · search.js

KẾT QUẢ

9,6×

FTS có GIN so với quét tuần tự



“nhan kim cuog solitare” → Nhẫn kim cương vàng 18K Solitaire (similarity 0,44)

Không cần Elasticsearch

3. pgvector – gợi ý sản phẩm tương tự

BÀI TOÁN Khách xem nhẫn kim cương vàng trắng → gợi ý sản phẩm "giống": cùng loại, cùng chất liệu, cùng đá. Quy tắc "cùng danh mục" quá thô.

GIẢI PHÁP: EMBEDDING 256 CHIỀU + INDEX HNSW

```
embedding vector(256)

CREATE INDEX idx_products_embedding
  ON products USING hnsw
  (embedding vector_cosine_ops);

-- 8 sản phẩm gần nhất (<=> = cosine)
SELECT p.name,
       1 - (p.embedding <=> me.embedding) sim
FROM products p,
     (SELECT embedding FROM products
       WHERE slug = $1) me
ORDER BY p.embedding <=> me.embedding
LIMIT 8;
```

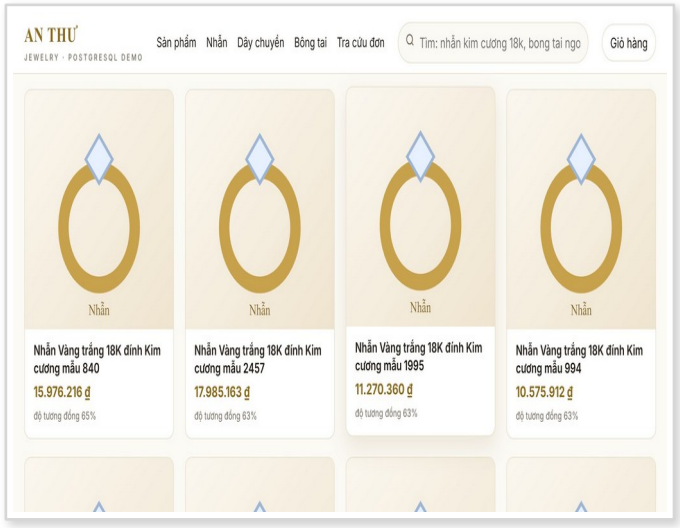
002 · 004 · catalog.js

- Embedding demo = feature hashing từ tên, danh mục, JSONB (không cần API ngoài); thay bằng OpenAI / CLIP chỉ đổi 1 hàm

KẾT QUẢ

7,4×

HNSW 0,23 ms so với 1,7 ms
tính toàn bộ 3.033 vector



4. Partitioning theo tháng + BRIN

BÀI TOÁN

Bảng đơn hàng chỉ tăng: sau vài năm hàng triệu dòng, nhưng nghiệp vụ hằng ngày chỉ chạm đơn gần đây. Index lớn, VACUUM chậm, xoá đơn cũ phải DELETE hàng loạt.

GIẢI PHÁP

```
CREATE TABLE orders ( ...,
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2026_10
  PARTITION OF orders
  FOR VALUES FROM ('2026-10-01')
               TO ('2026-11-01');

CREATE INDEX ... USING BRIN (created_at);
```

003 · 004_indexes.sql



KẾT QUẢ TRÊN 1 TRIỆU ĐƠN

1 / 37

partition được quét nhờ pruning
3,3 ms so với 66 ms (20×)

888 kB

BRIN so với 23 MB B-tree
cùng cột created_at (26×)

- Lưu trữ đơn cũ = DETACH 1 partition
- Tháng mới: ensure_orders_partition()
- VACUUM / index chỉ trên partition nhỏ

5. Transaction + FOR UPDATE – chống bán quá kho

BÀI TOÁN Nhẫn Solitaire còn đúng 1 chiếc, 2 khách bấm “Đặt hàng” cùng lúc. Nếu app chỉ SELECT stock rồi UPDATE, cả hai cùng thấy stock = 1 → bán 2 chiếc.

GIẢI PHÁP: 1 HÀM PL/PGSQL = 1 TRANSACTION

```
FOR r IN
  SELECT p.id, p.stock, i.qty
  FROM jsonb_to_recordset(p_items) i
  JOIN products p ON p.id = i.product_id
  ORDER BY p.id    -- cùng thứ tự, không deadlock
  FOR UPDATE OF p -- giao dịch 2 phải CHỜ ở đây
LOOP
  IF r.stock < r.qty THEN
    RAISE EXCEPTION 'OUT_OF_STOCK: %', r.stock;
  END IF;
END LOOP;
-- INSERT orders, order_items; UPDATE stock
-- CHECK (stock >= 0) là lớp bảo vệ cuối
```

005_order_transaction.sql

Giao dịch 2 chờ giao dịch 1 COMMIT → đọc stock đã trừ → OUT_OF_STOCK → không bao giờ bán quá kho

KẾT QUẢ (PGBENCH)

5 / 50

50 client cùng mua khi kho = 5
đúng 5 đơn, 45 bị OUT_OF_STOCK

1.090

đơn / giây, 30 client đồng thời
trên CSDL 1 triệu đơn

- API chỉ gọi SELECT place_order($1,$2)
- Huỷ đơn: cancel_order() hoàn kho cùng transaction
- Mọi thay đổi kho ghi vào stock_movements

6. Materialized view + LISTEN/NOTIFY

BÀI TOÁN

Dashboard cần doanh thu 30 ngày, top sản phẩm… tính trên 1,5 triệu dòng chi tiết mỗi lần mở là lãng phí. Admin muốn biết NGAY khi hàng sắp hết, không F5.

GIẢI PHÁP

```
CREATE MATERIALIZED VIEW mv_sales_daily AS
SELECT created_at::date AS day,
       count(*), sum(total) AS revenue
FROM orders WHERE status <> 'cancelled'
GROUP BY 1;
REFRESH MATERIALIZED VIEW CONCURRENTLY
  mv_sales_daily;    -- không chặn đọc

-- Trigger: stock < 3 → pg_notify('low_stock')
-- Node.js: LISTEN low_stock → SSE → admin
```

006 · 005 · events.js

KẾT QUẢ

1.000×

doanh thu 30 ngày: 0,018 ms
so với 19 ms tính trực tiếp

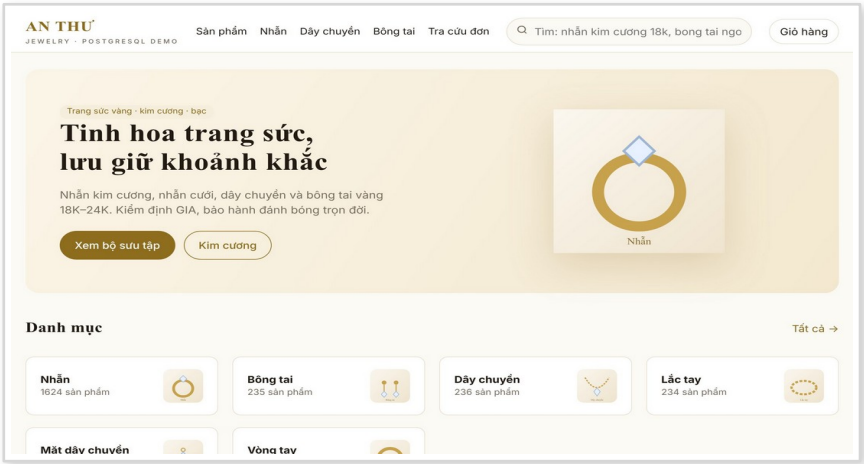
< 0,1 s

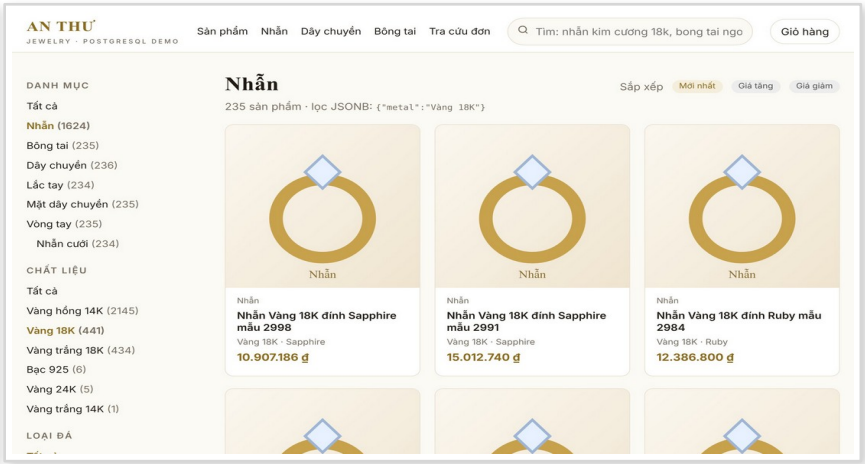
cảnh báo tới admin
không F5, không Redis

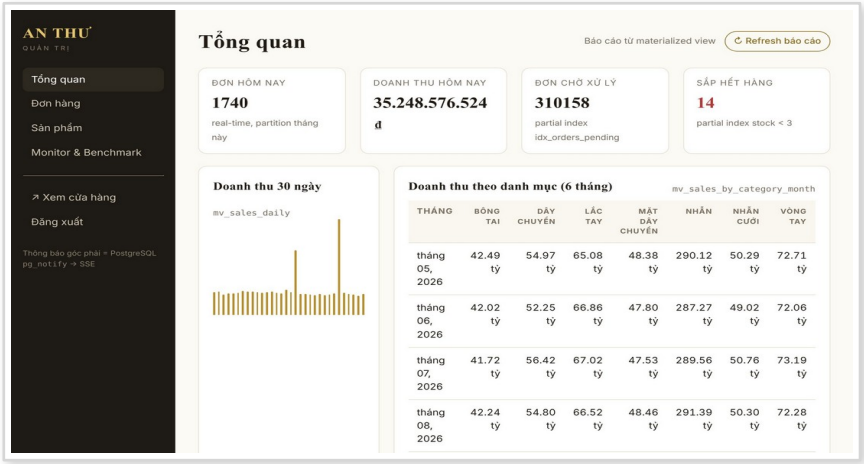


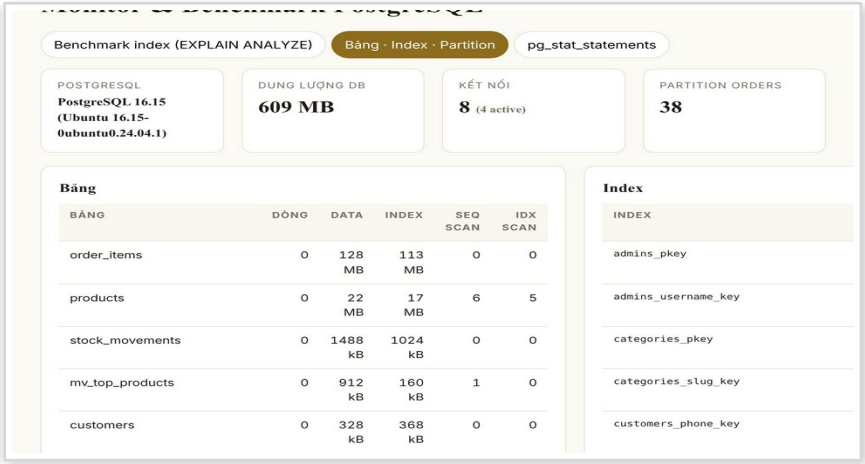


Ứng dụng: trang bán hàng và trang quản trị









Trang chủ · Lọc theo JSONB · Dashboard (matview + NOTIFY) · Monitor & Benchmark (EXPLAIN có/không index)

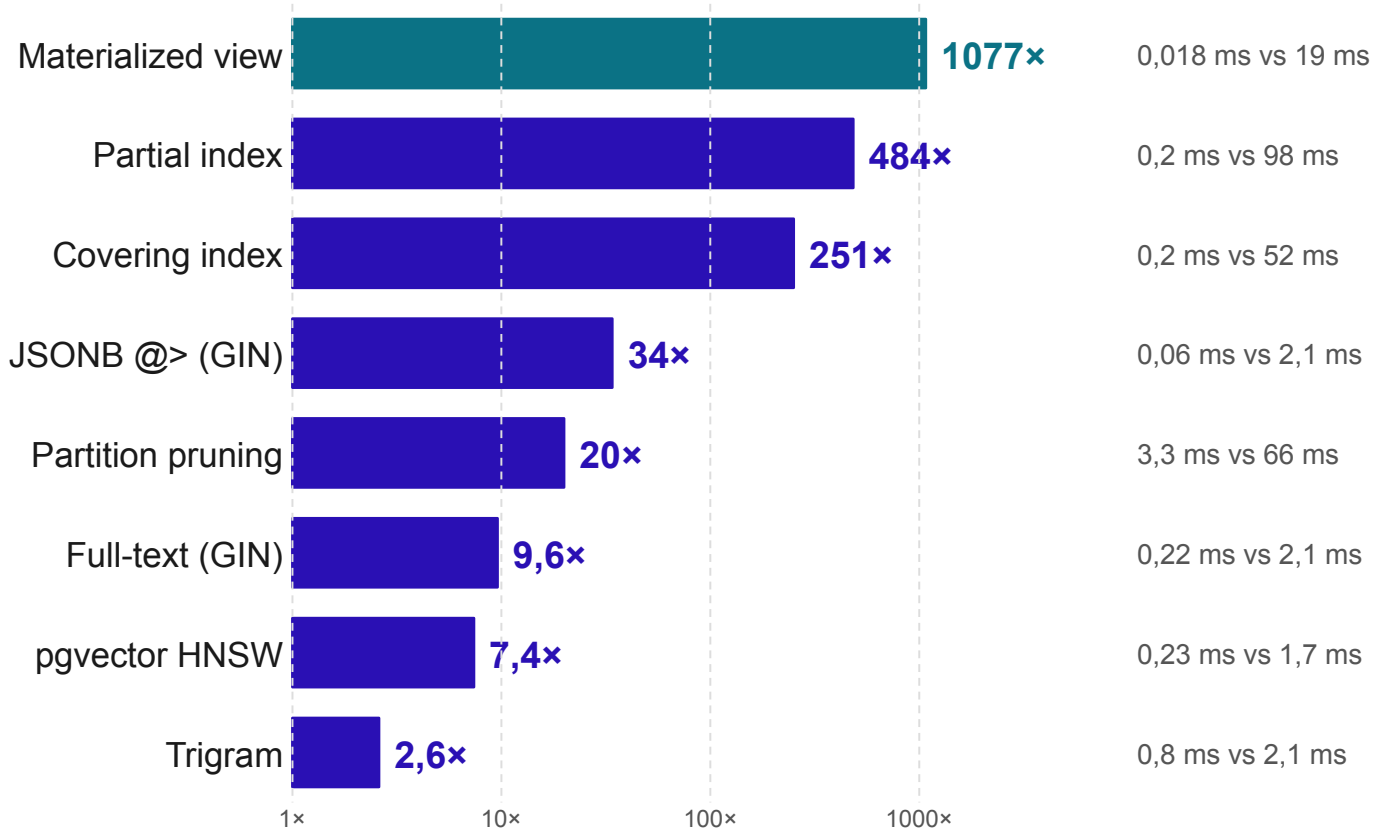
Demo trực tiếp bằng DBeaver

1. JSONB: lọc attributes @> {metal, stone} → GIN index
2. Full-text “nhan kim cuong 18k” · trigram “nhan kim cuog solitare”
3. pgvector: 8 sản phẩm giống nhẫn NK-001 (ORDER BY embedding <=> …)
4. Partition: EXPLAIN chỉ quét 1/37 partition · BRIN 888 kB vs B-tree 23 MB
5. place_order() 2 lần trên chiếc cuối → lần 2 báo OUT_OF_STOCK
6. Matview 0,02 ms vs JOIN 1,5 triệu dòng 1,1 s · UPDATE stock → NOTIFY

Kết nối: localhost:5432 · db jewelry · postgres / postgres — mở db/demo/demo_queries.sql, chọn khối lệnh, Cmd+Enter
Web: localhost:3000 · Admin → Monitor & Benchmark chạy EXPLAIN có/không index ngay trên giao diện

Benchmark: có index nhanh hơn bao nhiêu?

SỐ LẦN NHANH HƠN SO VỚI KHÔNG INDEX (1 TRIỆU ĐƠN, TRUNG VỊ 5 LẦN)



1 tr
đơn hàng
37 partition

1.090
đơn / giây
30 client

5 / 50
không bán
quá kho

Công cụ đo npm run bench — 12 kịch bản có/không index × 5 lần lấy trung vị, tắt index bằng SET LOCAL enable_… = off, kết quả ghi vào bench-report.md

Kết luận

Một hệ thống thay nhiều hệ thống

PostgreSQL = CSDL quan hệ + đa mô hình: thay cho search engine, cache, hàng đợi và vector DB trong phạm vi cửa hàng.

6 tính năng × 6 nghiệp vụ, đều có số đo

Trên 1 triệu đơn: nhanh hơn 2,6× – 1.000×; 50 client đồng thời không bán quá kho.

Hạn chế

Embedding feature hashing; chưa có từ điển tiếng Việt; chưa tinh chỉnh tham số PostgreSQL.

Hướng phát triển

Embedding CLIP theo ảnh; pg_cron tự tạo partition & refresh; replication đọc/ghi tách biệt.

Cảm ơn cô và các bạn đã lắng nghe!